

Sistema de *curtidas*

Comentar exige tempo; curtir leva um clique. É o jeito mais simples de o leitor dizer "gostei", e a primeira métrica que a autora vai acompanhar. Parece só um coração com um número, mas por trás há decisões de verdade: quantas vezes cada um curte, quem vê quem curtiu, e o que acontece com o número quando alguém clica cinco vezes seguidas.

Uma curtida por pessoa

Só artigos publicados

Total público, lista privada

Botão que responde na hora

Chave composta

Função de contagem

Formação Claude Code 2026 · Baseado em docs.lovable.dev (Connect to Supabase) e supabase.com/docs (Row Level Security, Database Functions), consultados em setembro de 2026 · Telas: reprodução ilustrativa

Seis decisões *antes do coração*

PERGUNTA	DECISÃO NO MEU BLOG	POR QUÊ
Quem curte?	Só quem está logado	Sem conta, qualquer um infla o número recarregando a página
Quantas vezes?	Uma por pessoa por artigo; clicar de novo descarte	O número precisa significar "pessoas que gostaram"
Em quê?	Só em artigos publicados	Rascunho não existe para o público (aula 03)
A autora curte o próprio?	Não	Ela sempre gostaria; o número perderia valor
Quem vê quem curtiu?	Ninguém além da própria pessoa; o total é público	Curtir é um gesto leve; não deve expor o leitor
Tempo real?	Não: o total se atualiza ao recarregar	Ver o número mudar sozinho não ajuda quem lê; nos comentários, sim

Curtir comentários fica fora deste módulo. A mesma receita desta aula serviria, trocando o artigo pelo comentário.

A tabela *curtidas*

COLUNA	REGRA
<code>artigo_id</code>	Ligada a artigos, <i>on delete cascade</i>
<code>user_id</code>	Padrão <code>auth.uid()</code> ; ligada a auth.users, <i>on delete cascade</i>
<code>criado_em</code>	Data da curtida (as aulas 09 e 10 usam)

CHAVE COMPOSTA: ARTIGO + USUÁRIO

A chave da tabela é o **par** (artigo_id, user_id). O banco simplesmente não aceita duas linhas com o mesmo par. "Uma curtida por pessoa" deixa de ser uma regra que a tela tenta seguir e vira algo impossível de quebrar.

POLÍTICA	REGRA
SELECT	Só as próprias: <code>user_id = auth.uid()</code>
INSERT	Logado, próprio <code>user_id</code> , artigo publicado e de outra pessoa
DELETE	Só as próprias
UPDATE	Nenhuma: curtida não se edita, se desfaz

Curtir é um INSERT; descurtir, um DELETE. Não existe coluna "curtido sim/não" para alguém alterar.

Contar *sem expor*

Se cada leitor só lê as próprias curtidas, como a tela mostra "25"? Uma **função no banco** faz a conta e devolve só números. Ela enxerga todas as linhas, mas entrega apenas o total e se *você* curtiu.

CURTIDAS_RESUMO (IDS)	DETALHE
Recebe	Uma lista de artigos: um só (página do artigo) ou os 9 da home
Devolve	Para cada artigo: <code>total</code> e <code>eu_curti</code>
Filtra	Só artigos publicados, ou da própria autora
Roda como	<i>security definer</i> , como <code>tem_papel</code> ; nunca devolve quem curtiu
Uma chamada para a lista inteira, não uma por card. Nove cards com nove chamadas deixam a home lenta à toa.	

CONTAR NA HORA OU GUARDAR UM CONTADOR?

Uma coluna `total_curtidas` em artigos seria mais rápida em blogs enormes, mas a autora pode atualizar os próprios artigos. Sem uma trava de coluna (aula 06), ela inflaria o número. Para um blog pessoal, contar na hora, com índice em `artigo_id` , é rápido e não tem esse risco.

Guardar cada curtida com data, em vez de só um número, é o que permite ao painel da aula 09 mostrar "curtidas nos últimos 30 dias" e à aula 10 achar os artigos **em alta**.

O pedido *do banco*

Pedido 1 (Plan mode) - banco:

1. Tabela curtidas: artigo_id (ligada a artigos, on delete cascade), user_id (padrão auth.uid(), ligada a auth.users, on delete cascade) e criado_em.
2. Chave primária composta (artigo_id, user_id); índice em artigo_id.
3. RLS: SELECT e DELETE só das próprias; INSERT logado, próprio user_id, apenas em artigo publicado e que não seja do próprio usuário. Sem UPDATE.
4. Função curtidas_resumo(ids uuid[]), security definer: para cada artigo publicado (ou do próprio autor), devolve total e eu_curti. Nunca devolve quem curtiu.

Explique o plano antes de aplicar.

NO PLANO, CONFIRA

- ✓ A chave é o **par**, não um id solto
- ✓ O INSERT confere publicado e "não é meu"
- ✓ Nenhuma política deixa ler as curtidas dos outros
- ✓ A função recebe uma **lista** de artigos
- ✓ Rascunhos ficam fora da contagem pública

Se o plano trouxer uma coluna `id` e deixar a unicidade para a tela, peça a chave composta. É ela que segura o duplo clique.

Responder *na hora*

Esperar o banco para pintar o coração deixa o clique "mole". O padrão chamado **atualização otimista** inverte a ordem: a tela muda primeiro e confirma depois.

- 1 Clique: o coração se pinta e o total sobe 1, **na hora**.
- 2 Em paralelo, a tela manda o INSERT (ou o DELETE, para descurtir).
- 3 Deu certo: nada muda. Deu erro: a tela **volta ao estado anterior** e mostra um aviso curto.
- 4 Enquanto o pedido não volta, o botão ignora novos cliques.

E O DUPLO CLIQUE?

Se, mesmo assim, dois INSERTs chegarem, o segundo esbarra na chave composta. A tela deve tratar esse erro ("já existe") como sucesso: a pessoa queria curtir, e curtido está.

ACESSIBILIDADE

O botão diz o estado em texto ("Curtir" / "Curtido") e informa aos leitores de tela que está pressionado (`aria-pressed`). Cor sozinha não basta.

O pedido *das telas*

Pedido 2 - curtidas nas telas:

- Página do artigo: botão "Curtir - N" ao lado de Compartilhar, abaixo do título; curtido fica "Curtido - N", coração cheio, fundo rosado, aria-pressed.
- Atualização otimista: muda na hora, desfaz se der erro; ignora cliques enquanto espera; "já existe" = sucesso.
- Visitante: o clique abre um balão "Gostou do artigo?" com Entrar e Criar conta; depois do login, volta ao mesmo artigo, como nos comentários.
- Autora no próprio artigo: vê o total, sem botão de curtir.
- Home e /tag: cada card mostra coração e total, sem clique. Use UMA chamada a curtidas_resumo para a página inteira.
- Números no formato brasileiro (1.234).

POR QUE NÃO CURTIR PELO CARD

Curtir pressupõe ter lido. No card, o coração só informa; a decisão fica para o fim da leitura.

Esconder o botão da autora é conforto. Quem impede de verdade que ela curta o próprio artigo é a política de INSERT.

O coração *na página do artigo*



Leitor B logado, depois de curtir · reprodução ilustrativa com dados de teste

ESTADO CLARO

"Curtido", coração cheio e fundo rosado: três sinais, não só a cor.

NO LUGAR CERTO

Junto de Compartilhar, na linha da autora: as duas ações de quem gostou.

NÚMERO HONESTO

25 pessoas diferentes. A chave composta garante.

O total nos cards, *o convite para o visitante*



Home para visitante · reprodução ilustrativa

O terceiro card mostra "0". Poderia esconder o número zerado; aqui ficou visível, para o leitor ver que é um artigo novo, e não um erro.



Visitante clica em Curtir

CONVITE, NÃO BLOQUEIO

O visitante vê o total e pode clicar. Em vez de um erro, recebe um convite e volta ao mesmo artigo depois de entrar.

Tentando *inflar o número*

1 B curte e descurte: o total sobe e desce; recarregue e confira que ficou certo.

2 B clica cinco vezes bem rápido: no fim, uma curtida ou nenhuma, nunca duas. Confira no **Table Editor**.

3 B, C e D curtem: total 3. Na home, o card mostra 3.

4 Autora Teste abre o próprio artigo: vê o total, sem botão.

5 Pergunte no Chat mode se C consegue ler quem curtiu, curtir um rascunho ou o próprio artigo: não.

6 Numa janela anônima, clique em Curtir: balão; entre como D e volte ao mesmo artigo.

REGISTRO DOS TESTES **EXEMPLO**

Meu Blog · curtidas

✓ curtir e descurtir	ok
✓ 5 cliques rápidos	1 linha no máximo
✓ B, C e D curtem	total 3
✓ card da home	3
✓ autora no próprio artigo	sem botão
✓ ler quem curtiu	não
✓ curtir rascunho	recusado
✓ curtir o próprio artigo	recusado
✓ visitante → login → volta	ok

bookmark: "curtidas v1"

No Table Editor você vê todas as linhas de **curtidas**, porque o painel é admin. É ali que o teste do duplo clique fica provado.

Erros comuns *com curtidas*

Unicidade só na tela. Um duplo clique ou uma chamada à API cria a segunda curtida. A chave composta resolve no banco.

Contador que a autora pode editar. Uma coluna de total em artigos, sem trava, vira número inventado.

Leitura pública das curtidas. Para mostrar o total, alguém libera o SELECT, e qualquer um descobre o que cada leitor curtiu.

Uma chamada por card. Nove cards, nove idas ao banco. Uma lista, uma chamada.

Otimista sem desfazer. O coração fica pintado mesmo quando o banco recusou, e a pessoa acha que curtiu.

Só cor para o estado. Quem não distingue o vermelho não sabe se curtiu. Texto e `aria-pressed`.

Teste o que *ficou*

1. O que garante uma curtida por pessoa por artigo?

A) O botão desabilitado

B) A chave composta (artigo_id, user_id)

C) O tempo real

D) O Knowledge

B. O banco recusa o par repetido, venha de onde vier.

2. Como a tela mostra o total se ninguém lê as curtidas dos outros?

A) Liberando o SELECT para todos

B) Uma função no banco devolve só o total e o eu_curti

C) Guardando o número no navegador

D) Não mostra

B. A função vê as linhas, mas entrega só números.

3. Na atualização otimista, o que acontece se o banco recusar?

A) Nada

B) A tela volta ao estado anterior e avisa

C) Tenta de novo para sempre

D) Recarrega a página

B. Mudar na hora só é honesto se desfizer no erro.

4. Por que guardar cada curtida com data, e não só um número?

A) Ocupa menos espaço

B) Permite métricas por período e o "em alta" das próximas aulas

C) O Supabase exige

D) Para mostrar quem curtiu

B. Um total não diz *quando*; as linhas dizem.

○ RECAPITULANDO

Um clique, *um número honesto*

Leitores logados curtem artigos publicados de outras pessoas, uma vez cada, garantido pela chave composta. As curtidas são privadas; o total vem de uma função que só devolve números, numa chamada para a página inteira. O botão responde na hora e desfaz no erro, e o visitante recebe um convite para entrar. Cada curtida tem data: a matéria-prima do painel e do "em alta". Bookmark "curtidas v1".

curtidas

Chave (artigo_id, user_id)

curtidas_resumo

Atualização otimista

aria-pressed

Sem curtir o próprio

Próxima aula → Sistema de favoritos

Fontes: docs.lovable.dev (Connect to Supabase) e supabase.com/docs (Row Level Security e Database Functions), consultados em setembro de 2026. As telas são reproduções ilustrativas feitas para o curso, com contas e dados de teste.